

Page Object Model (POM) in Selenium

Best Practices for Scalable Automation

◆ How to Implement POM in Selenium with Java

There are **two ways** to implement POM:

- 1 Without Page Factory (Manual Approach)
 - 2 With Page Factory (Optimized Approach)
-

1 Implementing POM Without Page Factory

📌 In this approach, we manually define locators using **By** and initialize them in a constructor.

◆ Login Page Class (**LoginPage.java**)

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class LoginPage {
    WebDriver driver;

    // Define locators using By
    By username = By.id("username");
    By password = By.id("password");
    By loginButton = By.id("loginBtn");

    // Constructor to initialize WebDriver
    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    // Actions
    public void enterUsername(String user) {
        driver.findElement(username).sendKeys(user);
    }

    public void enterPassword(String pass) {
        driver.findElement(password).sendKeys(pass);
    }

    public void clickLogin() {
        driver.findElement(loginButton).click();
    }
}
```

◆ Test Class (**LoginTest.java**)

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class LoginTest {
    public static void main(String[] args) {
        System.setProperty("webdriver.chrome.driver", "path/to/chromedriver");
        WebDriver driver = new ChromeDriver();

        driver.get("https://example.com/login");

        // Create object of LoginPage and perform actions
        LoginPage login = new LoginPage(driver);
        login.enterUsername("testuser");
        login.enterPassword("password123");
        login.clickLogin();

        driver.quit();
    }
}
```

✓ **Pros:** Full control over element initialization.

✗ **Cons:** More boilerplate code, manual element handling.

2 Implementing POM With Page Factory

📌 Page Factory uses `@FindBy` annotations to define elements and `PageFactory.initElements(driver, this);` to initialize them.

◆ Login Page Class (`LoginPage.java`)

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.PageFactory;

public class LoginPage {
    WebDriver driver;

    // Using @FindBy annotation for elements
    @FindBy(id = "username") WebElement username;
    @FindBy(id = "password") WebElement password;
    @FindBy(id = "loginBtn") WebElement loginButton;

    // Constructor to initialize PageFactory elements
    public LoginPage(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this);
    }

    // Actions
    public void enterUsername(String user) {
        username.sendKeys(user);
    }

    public void enterPassword(String pass) {
        password.sendKeys(pass);
    }

    public void clickLogin() {
        loginButton.click();
    }
}
```

◆ Test Class (**LoginTest.java**)

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class LoginTest {
    public static void main(String[] args) {
        System.setProperty("webdriver.chrome.driver", "path/to/chromedriver");
        WebDriver driver = new ChromeDriver();

        driver.get("https://example.com/login");

        // Create object of LoginPage and perform actions
        LoginPage login = new LoginPage(driver);
        login.enterUsername("testuser");
        login.enterPassword("password123");
        login.clickLogin();

        driver.quit();
    }
}
```

✅ **Pros:** Clean code, better maintainability.

❌ **Cons:** Slightly less control over WebElement initialization.

📌 Best Practices for POM Implementation

- ◆ Keep **locators and methods separate** from test scripts.
- ◆ Use **descriptive method names** for better readability.
- ◆ Implement **Base Page Class** for common methods like `waitForElement()`, `clickElement()`, etc.
- ◆ Use **logging and reporting** for better debugging and test insights.

If you're building a **robust Selenium framework**, **POM + Page Factory** is the way to go! 🎯